

Comparative Study of Single-Game and Multi-Game RL Agents

Benedikt Pilger
LMU Munich

Benedikt.Pilger@campus.lmu.de

Ilir Hajrullahu
LMU Munich

I.Hajrullahu@campus.lmu.de

Abstract

This project investigates the application of model-free reinforcement learning to two classic games—Snake and Pac-Man—and explores the feasibility of training a single agent capable of playing both. We implement and compare two approaches: a value-based Deep Q-Learning agent and a policy-gradient Proximal Policy Optimization (PPO) agent. Each method is first trained independently on both games using custom Gym wrappers, convolutional neural networks, and carefully designed reward shaping. We then introduce a multi-task framework where a shared convolutional encoder processes inputs from both games, and evaluate several training regimes, including naïve mixing, curriculum learning, and multi-head architectures. Performance is assessed in terms of learning speed, sample efficiency, and final scores. Results show that PPO significantly outperforms Q-Learning in sparse-reward, high-dimensional settings like Pac-Man. Moreover, multi-head architectures with shared feature extractors partially mitigate task interference and enable modest cross-game generalization. We conclude with recommendations for improving stability and transfer through task-conditioned networks and extended curricula.

1. Introduction

This report documents our project for the “Advanced and Applied Machine Learning” practical, titled “Cross-Game Reinforcement Learning Agents for Snake and Pac-Man.” We focus on two classes of model-free RL algorithms: tabular Q-Learning for low-dimensional, discrete states, and Proximal Policy Optimization (PPO) for high-dimensional inputs processed by convolutional neural networks [3, 4].

Our specific objectives are:

- Train and evaluate individual agents for each game (Snake and Pac-Man) using both Q-Learning and PPO, establishing baseline performance.
- Develop a single multi-task agent capable of playing both games by sharing a common policy architecture.
- Compare sample efficiency, stability, and final perfor-

mance across algorithms and single- versus multi-task settings.

Key research questions include:

- How do value-based and policy-gradient methods differ in learning dynamics and ultimate proficiency on Snake and Pac-Man?
- Can a shared PPO policy effectively transfer knowledge between two distinct environments, or does multi-task training degrade specialization?
- What insights into representation learning and exploration emerge from cross-game experiments?

Beyond academic interest, multi-task RL has practical applications in transfer learning, lifelong learning, and general-purpose game AI, where a single agent must adapt to varied tasks.

The remainder of this report is organized as follows. We review prior work in RL and game benchmarks in [Section 2](#). Our environment designs and algorithm implementations are detailed in [Section 3](#). Experiments on single-game agents appear in [Section 4](#), and multi-task experiments in [Section 5](#). Finally, we summarize findings and outline future directions in [Section 6](#).

2. Related Work

Reinforcement learning has its theoretical foundations in Sutton and Barto’s textbook, which formalizes Markov decision processes, value iteration, and policy iteration, and introduces core concepts such as the exploration–exploitation trade-off and temporal-difference learning [4]. Watkins and Dayan’s Q-learning algorithm later demonstrated that an off-policy temporal-difference method could converge to the optimal action-value function in the tabular setting without requiring an explicit model of the environment [5].

The advent of Deep Q-Networks (DQN) by Mnih et al. marked a breakthrough in combining Q-learning with deep neural networks: by training a convolutional network directly on raw pixel inputs and using techniques like experience replay and target networks, DQN achieved human-level performance on Atari 2600 games, spurring numerous extensions and improvements [2].

Proximal Policy Optimization (PPO), introduced by Schulman et al., is a modern policy-gradient algorithm that simplifies trust-region methods by clipping the policy update ratio. PPO achieves strong performance across a variety of control benchmarks without the complexity of second-order optimization [3].

For practical guidance, Lapan’s hands-on book provides step-by-step implementations of deep RL algorithms—including tabular Q-learning, DQN, and PPO—using PyTorch and OpenAI Gym, making these methods accessible to practitioners [1].

3. Methodology

3.1. Proximal Policy Optimization (PPO)

Our PPO agent is built on Stable-Baselines3 and interacts with the Snake environment through a custom Gym wrapper. The environment presents observations as three binary channels—snake head, snake body, and food—stacked to form a $3 \times H \times W$ tensor with values normalized to $[0, 1]$. To shape learning, each time step carries a penalty of -0.01 , collisions incur -100 , and eating food yields $+100$. Additionally, when the snake’s head moves closer to the food we grant a bonus of $+1$. We support two initialization modes: in the fixed-start setup the snake always begins at the grid center facing downward, while in the random-start setup its initial position and heading are sampled uniformly across the grid. Episodes are capped at 1000 steps via Gym’s TimeLimit wrapper.

The policy network uses a custom convolutional feature extractor, CustomCNN, comprising two 3×3 convolutional layers (32 and 64 filters) with ReLU activations, followed by flattening and a fully connected layer producing a 128-dimensional embedding. This feeds into SB3’s built-in CnnPolicy (with image normalization disabled). We train using PPO with a learning rate of 2.5×10^{-4} , a rollout length of 512 steps, batch size 64, three gradient epochs per update, and an entropy coefficient of 0.01. Training is performed on a Nvidia GTX1070 with 8GB of VRAM. To improve sample efficiency, experience is collected in parallel from four environments via SubprocVecEnv. Performance is monitored during training with an EvalCallback that runs 10 deterministic test episodes every 250,000 timesteps and logs the mean and standard deviation of episode rewards.

We conduct a systematic budget sweep over six training durations: 100,000; 250,000; 500,000; 1,000,000; 2,000,000; and 3,000,000 steps. For each duration, we train separate agents under both fixed-start and random-start initializations, then evaluate according to the protocol above.

3.2. Deep-Q-Learning

The agent in this work is trained using the Deep Q-Learning (DQN) framework, which combines traditional Q-learning with deep neural networks as function approximators. Specifically, a convolutional neural network (CNN) is used to approximate the action-value function $Q(s, a; \theta)$, mapping a high-dimensional grid-based representation gem environments to Q-values for each discrete action. This allows the agent to generalize across similar states and learn spatial features.

To improve stability during training, the agent incorporates two key DQN enhancements: experience replay and a target network. A fixed-size replay buffer stores transitions (s, a, r, s') sampled randomly to break temporal correlations, while a separate target network $Q(s, a; \theta^-)$ is used to compute the temporal difference targets. This architecture enables the agent to learn an optimal policy over time by minimizing the Bellman loss between predicted and target Q-values.

Convolutional Q-Network Architecture

The Q-network receives a 21×19 single-channel grid input representing the current state of the game environments. The network is composed of three convolutional layers with ReLU activations and batch normalization, followed by global average pooling and a fully connected output layer. The architecture is as follows:

- **Input:** $\mathbf{x} \in \mathbb{R}^{1 \times 21 \times 19}$
- **Conv1:** 32 filters, 3×3 kernel, padding 1, stride 1

$$\mathbf{z}_1 = \text{ReLU}(\text{BN}_1(\text{Conv2D}(\mathbf{x}, 32, 3, \text{stride} = 1, \text{padding} = 1)))$$
- **Conv2:** 64 filters, 3×3 kernel, padding 1, stride 1

$$\mathbf{z}_2 = \text{ReLU}(\text{BN}_2(\text{Conv2D}(\mathbf{z}_1, 64, 3, \text{stride} = 1, \text{padding} = 1)))$$
- **Conv3:** 128 filters, 3×3 kernel, padding 1, stride 1

$$\mathbf{z}_3 = \text{ReLU}(\text{BN}_3(\text{Conv2D}(\mathbf{z}_2, 128, 3, \text{stride} = 1, \text{padding} = 1)))$$
- **Global Average Pooling:**

$$\mathbf{z}_4 = \text{GAP}(\mathbf{z}_3) \in \mathbb{R}^{128}$$
- **Output Layer:** Fully connected layer mapping to 4 actions (LEFT, RIGHT, UP, DOWN)

$$Q(s, a) = \mathbf{W} \cdot \mathbf{z}_4 + \mathbf{b}, \quad \mathbf{W} \in \mathbb{R}^{4 \times 128}$$

The network is optimized using mean squared error loss between the predicted Q-values and the target Q-values defined by the Bellman equation. The model supports n-step Q-learning and uses a target network to stabilize learning.

Bellman Update and Q-Learning

The Q-learning algorithm seeks to learn the optimal action-value function $Q^*(s, a)$, which estimates the expected cumulative future reward for taking action a in state s , and thereafter following the optimal policy. The core of Q-learning is the Bellman equation:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right] \quad (1)$$

In deep Q-learning, the Q-function is approximated by a neural network $Q(s, a; \theta)$ with weights θ . The update rule becomes:

$$L(\theta) = \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (2)$$

Here, θ^- are the weights of a target network that are periodically synchronized with θ , and $\mathcal{D}$ is the replay buffer containing past experiences.

Training Loop

The agent interacts with the environment in discrete episodes. Each step consists of:

1. Observing the current state s_t .
2. Selecting an action a_t via an ϵ -greedy or softmax exploration policy.
3. Executing a_t in the environment and receiving reward r_t and next state s_{t+1} .
4. Storing the tuple $(s_t, a_t, r_t, s_{t+1}, \text{done})$ in the replay buffer.
5. Sampling a mini-batch from the buffer and minimizing the Bellman loss $L(\theta)$.

Experience Replay and Target Networks

Experience replay is used to stabilize training by breaking correlations between sequential samples. The agent maintains a buffer $\mathcal{D}$ of the N most recent experiences. At each training step, it samples a random batch to update the Q-network.

To further stabilize training, a separate target network $Q(s, a; \theta^-)$ is used to compute the target Q-value, and its weights θ^- are periodically updated to match the current network’s weights θ every K steps.

This double-buffering strategy helps reduce the oscillations and divergence that often arise when using non-linear function approximators like deep networks in reinforcement learning.

Training

Training is performed on a Nvidia RTX4060Ti with 16GB of VRAM. To compare the agents the trained models are

Timesteps	Fixed Start	Random Start
100 000	492.8 $\pm$ 19.5	463.7 $\pm$ 130.3
250 000	490.0 $\pm$ 0.0	475.1 $\pm$ 102.4
500 000	491.0 $\pm$ 9.9	441.3 $\pm$ 181.4
1 000 000	492.1 $\pm$ 14.6	394.9 $\pm$ 233.8
2 000 000	491.5 $\pm$ 15.4	444.3 $\pm$ 160.9
3 000 000	491.5 $\pm$ 15.4	445.6 $\pm$ 161.9

Table 1. PPO on Snake: mean $\pm$ standard deviation of episode reward over 100 runs, for both fixed-start and random-start initializations.

stored after 1000, 2000, 5000 and 10000 games. These models are then evaluated with 100 runs per game per model.

4. Single Agent Experiments and Results

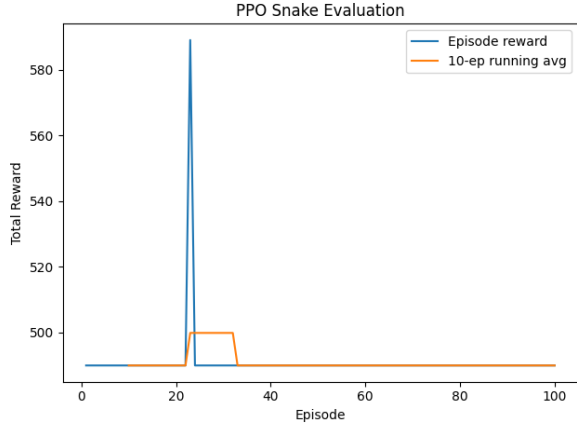
4.1. Snake (PPO)

Table 1 reports the mean and standard deviation of episode rewards for our PPO agent on Snake, evaluated over 100 test episodes at six different training budgets. Both fixed-start (centered) and random-start initializations are shown.

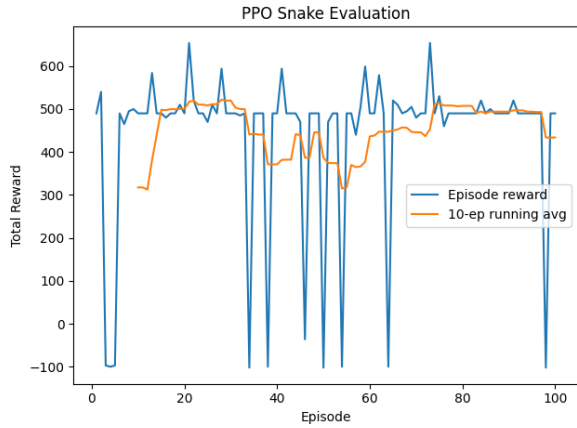
Figure 1a shows the 100-episode evaluation of the 500 k-step PPO policy under the fixed-start (centered) condition. Aside from one outlier episode around Episode 20—where the snake managed to collect six apples before the 1 000-step limit—the total reward per episode is nearly constant at about 490, corresponding to the optimal five-apple loop with only occasional extra pickups. The 10-episode running average briefly spikes to 500 when that lucky run enters the window, then immediately returns to 490. This near-flat performance confirms that, in a deterministic start setting, the policy has effectively memorized the shortest path to the food and repeats it almost flawlessly.

Figure 1b displays the evaluation under the random-start flag after the same 500 k steps. Here the episode rewards swing from -100 (immediate collisions) up to 600 (exceptional layouts) as each begins in a different corner and heading. The running average (orange) starts low (320) over the first dozen episodes—when the agent still misjudges novel starts—but climbs steadily as it encounters more varied initial states, settling around 450–500 by Episode 30. The remaining variability (peaks and deep troughs) reflects rare “corner-case” failures, such as starting directly against a wall with the food behind the snake. Overall, the random-start plot illustrates the extra sample complexity needed to generalize beyond a single scripted loop and highlights the policy’s progress in learning recovery strategies from arbitrary begins.

Across all budgets, the fixed-start configuration yields an overall stable average reward of roughly 492–493 with low



(a) Evaluation after training with deterministic start



(b) Evaluation after training with random start

Figure 1. Evaluation on 100 games after training Snake 500k timesteps with PPO

variance, corresponding to nearly five apples per episode. Random-start runs exhibit lower means and higher variance, improving up to 250 k steps before plateauing around 440–450 reward, indicating diminishing returns for generalization beyond that point.

4.2. Snake (Q Learning)

Reward Components

• Visitation Rewards:

- Revisiting a cell yields a small reward proportional to $\frac{1}{\text{visit count}} \times 0.5$.
- Visiting a new cell grants a bonus:

$$\text{bonus} = \max(0.05, 0.5 \times (1 - \frac{\text{frame iteration}}{300}))$$

Scaled further if fewer than 20 cells have been visited.

• Game Over Penalties:

- Collision or exceeding frame limit results in a -10 reward.

- Additional penalty of -5 if the player does not score within the first 15 frames.

• Food Collection Rewards:

- Eating food in the first n games grants a large reward to ensure a strong learning in the first steps:

$$30 \quad (\text{if } n_{\text{games}} < 300)$$

Otherwise:

$$15 + \text{time bonus} \times 5 + \text{streak bonus}$$

where

$$\text{time bonus} = \max(0, 1 - \frac{\text{frame iteration}}{200}), \quad \text{streak bonus} = \frac{\text{streak}}{25} + \text{score}$$

- **Movement Efficiency** For a better learning curve also the movement of the agent is considered this includes:

- Moving closer to the food: $+0.5$
- Moving away from the food: -0.2
- Staying close to borders: -0.1
- Repetitive movements (limited direction change): -0.2

• Stalling Penalties:

- Staying too long without eating food results in small penalties:

$$\text{if steps since last food} > 100 : -0.2$$

$$\text{if steps since last food} > 300 : -1 \quad \text{and game over}$$

• Action-Based Shaping:

- Moving straight: $+0.1$
- Turning: -0.02

All final rewards are clipped between $[-10, 10]$ to ensure numerical stability during learning.

Results and analysis

The evaluation results indicate that the model’s performance improves over time. Specifically, Figure 2 shows the reward per game has a rising trend after 6000 training games, suggesting that the agent is learning to make better decisions as training progresses.

In Figure 5 is shown that the action distribution appears balanced, with no single action being disproportionately favored. The total number of actions increases over time, this means the agent learns to survive longer.

Additionally, Figure 3 shows the average evaluation score steadily increases across training steps, further confirming the model’s improved competence.

Overall it can be assumed that with more training the agent would keep improving at this point since no plateau is reached.

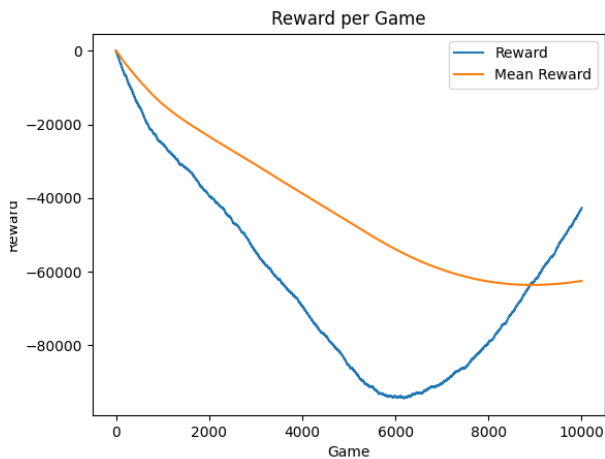


Figure 2. Reward per game and mean reward per game in training

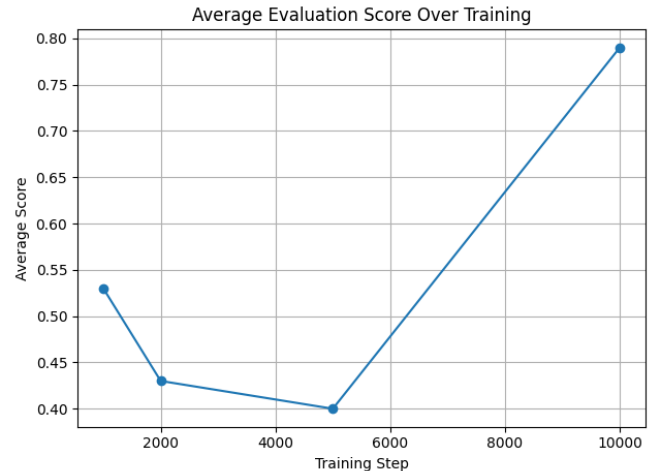


Figure 4. Average score over 100 evaluation games

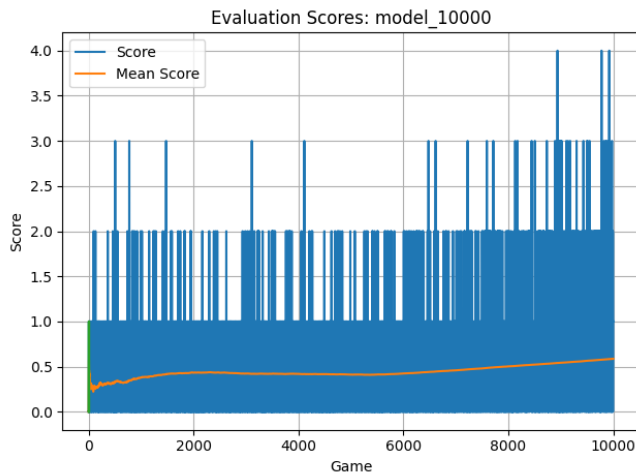


Figure 3. Score in each training game and average training score

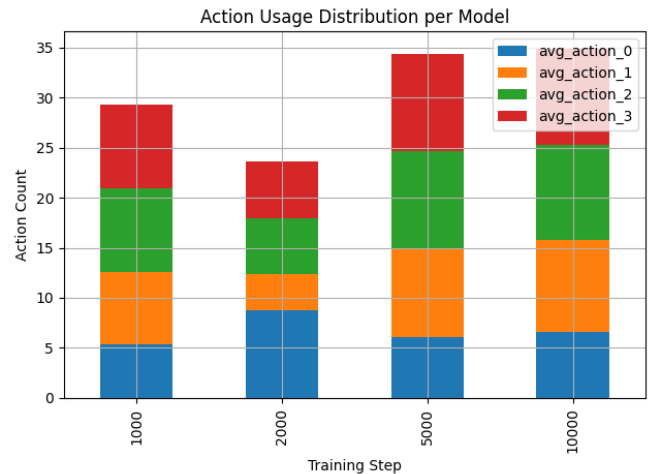


Figure 5. Average action distribution per game over 100 evaluation games

4.3. Pacman (PPO)

Table 2 summarizes the performance of our PPO-trained Pac-Man agent over six different training budgets, with each entry giving the mean and standard deviation of total episode reward computed over 100 evaluation runs.

Despite extensive training, the agent’s average score remains modest, hovering between approximately 21 and 33 points. Since each pellet is worth 10 points, this corresponds to the agent typically collecting only three or four pellets before either being caught by a ghost or reaching the environment’s time limit. Early in training (100K and 250K timesteps), performance is particularly unstable, with standard deviations exceeding 9 points, indicating that the policy has not yet discovered a reliable strategy and often fails to navigate even the closest pellets. As training progresses

Timesteps	Mean $\pm$ Std Dev
100 000	25.87 ± 9.56
250 000	21.19 ± 12.16
500 000	25.88 ± 9.81
1 000 000	26.28 ± 9.35
2 000 000	22.08 ± 12.11
3 000 000	33.36 ± 13.00

Table 2. PPO on Pac-Man: mean $\pm$ standard deviation of episode reward over 100 runs at various training budgets.

to half a million and one million timesteps, the mean reward barely improves, suggesting that the agent becomes entrenched in a local optimum—repeatedly executing the

same short path to nearby pellets without exploring the full maze. Interestingly, at three million timesteps we observe a more noticeable increase to 33.36 ± 13.00 ; this late uptick implies that with a sufficiently long training horizon, the policy can occasionally break out of its local strategy and reach more distant pellets, although variance remains high.

These results imply that, under the current pixel-to-grid representation and reward shaping, PPO struggles to perform the long-range planning required to clear a level. We assume that the sparse reward structure and large state space are key obstacles: once the immediate rewards from the closest pellets are exhausted, there is insufficient incentive for Pac-Man to navigate deeper into the maze. To address this, one could introduce a substantial “level-clear” bonus when all pellets are eaten, implement a curriculum that begins with fewer ghosts or simplified mazes, or enrich the observation with distance-to-nearest-pellet features. Moreover, experimenting with hierarchical or model-based extensions of PPO might provide the agent with better foresight in planning longer pellet-collection runs. Future work should therefore explore such modifications, as well as alternative architectures—such as attention-based networks—that can more effectively handle the combinatorial complexity of full-board Pac-Man strategies.

4.4. Pacman (Q Learning)

Reward Components

- **Pellet consumption:** When the agent moves to a tile containing a pellet, it receives a reward of +10. This encourages pellet-seeking behavior.
- **Energizer consumption:** Eating a power pellet (energizer) yields a reward of +50 and enables the agent to enter a frightened-ghost-chasing mode.
- **Eating ghosts:** If the agent collides with a frightened ghost, the ghost is respawned and the agent receives a high-value reward of +200, incentivizing strategic use of energizers.
- **Ghost collision (non-frightened):** If the agent collides with a non-frightened ghost, it loses a life and incurs a penalty of −300. If this results in all lives lost, the episode ends.
- **Game win:** Clearing all pellets and energizers results in a terminal reward of +1000, indicating successful completion of the level.

All final rewards are clipped between $[-10, 10]$ to ensure numerical stability during learning.

Results and analysis

Training scores are rising:

As Figure 6 demonstrates consistent improvement in training scores over time, indicating that the agent is learning to optimize its behavior in the environment.

Action distribution is learning to do only one action:

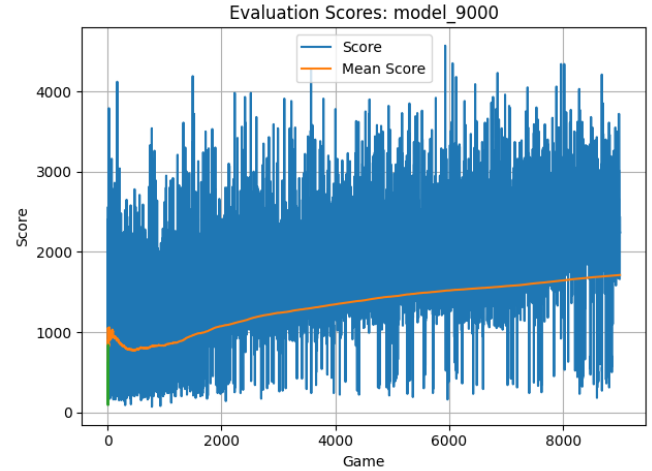


Figure 6. Score in each training game and average training score

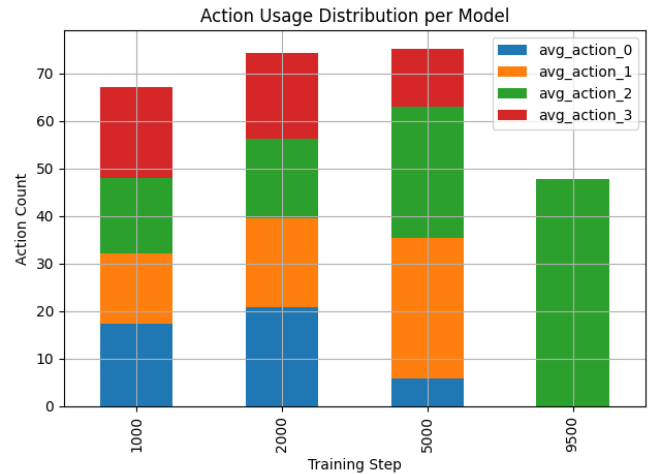


Figure 7. Average action distribution per game over 100 evaluation games

In Figure 7 is shown that the action distribution in early games is doing well, but with more training games the agent increasingly favors a single action.

Average score rises in the first 5000 games, then declines:

Figure 8 shows a peak in average performance around the 5000-game mark, followed by a noticeable decline.

Conclusion

While the model initially learns effectively, indicated by rising scores and action certainty, it eventually suffers from overfitting and reduced diversity in behavior.

The reason for this issue is a wrong coding of the ghosts movement which kept the ghosts stuck in a position or running in circles. The agent learned to only move upward,

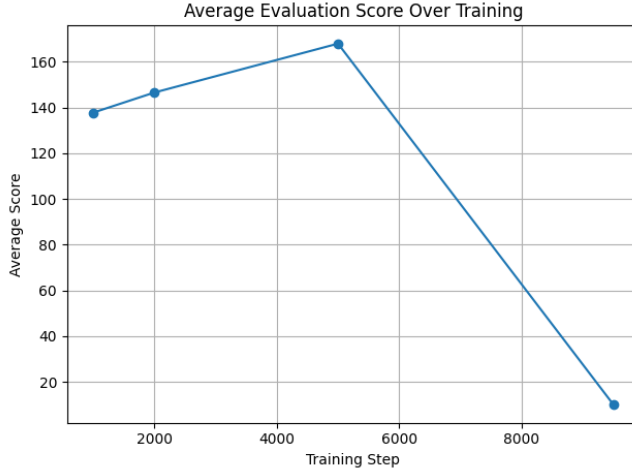


Figure 8. Average score over 100 evaluation games

Regime	Snake mean ± std	Pac-Man mean ± std
Naïve mixed, one head (100 K)	491.5 ± 11.1	39.6 ± 0.0
Naïve mixed, one head (250 K)	430.0 ± 102.4	39.6 ± 0.0
Curriculum (125 K → 125 K)	413.1 ± 200.4	39.8 ± 0.0
Multi-head (250 K → 250 K)	248.5 ± 294.5	89.6 ± 0.0

Table 3. Mean ± standard deviation of episode reward over 100 runs for Snake and Pac-Man under three multi-game PPO regimes.

which resulted in a stationary movement since the first up movement goes into a wall.

This issue was fixed for the combined agent but due to the limited time the pacman standalone agent couldnt be retrained.

5. Combined Agent Experiments and Results

5.1. PPO Approach

We trained a single PPO agent to play Snake and Pac-Man alternately, using eight headless environments and a shared CNN backbone, under three different multi-game regimes. Table 3 gives the 100-episode mean and standard deviation of total reward for each regime.

Naïve mixed: Using a single shared policy/value head, we alternated Snake and Pac-Man frames from the start. After 100 K timesteps, Snake retained its optimal five-apple loop ($\approx 491 \pm 11$) while Pac-Man remained at its trivial four-pellet baseline ($\approx 40 \pm 0$). By 250 K timesteps, Snake’s

performance degraded to $\approx 430 \pm 102$, indicating destructive gradient interference.

Curriculum: We then pre-trained on Pac-Man alone for 125 K timesteps before 125 K of mixed training, still with a single head. Pac-Man showed no appreciable gain ($\approx 40 \pm 0$), while Snake’s mean fell further to $\approx 413 \pm 200$, confirming that a shared head cannot prevent cross-task conflicts.

Multi-head: Finally, we added a 7th “task-ID” channel (0 for Snake, 1 for Pac-Man) and two separate policy/value heads atop the shared CNN. Under a 250 K → 250 K (pretrain → mixed) schedule, Pac-Man broke free—achieving $\approx 89.6 \pm 0$ —while Snake averaged $\approx 248.5 \pm 294.5$, showing that separate heads eliminate final-layer interference but cannot fully stabilize shared features.

Figure 9 illustrates the episode-by-episode rewards for the final multi-head run: Pac-Man (orange) quickly reaches and maintains its improved mean, whereas Snake (blue) exhibits high volatility, oscillating between near-failure and near-optimal loops.

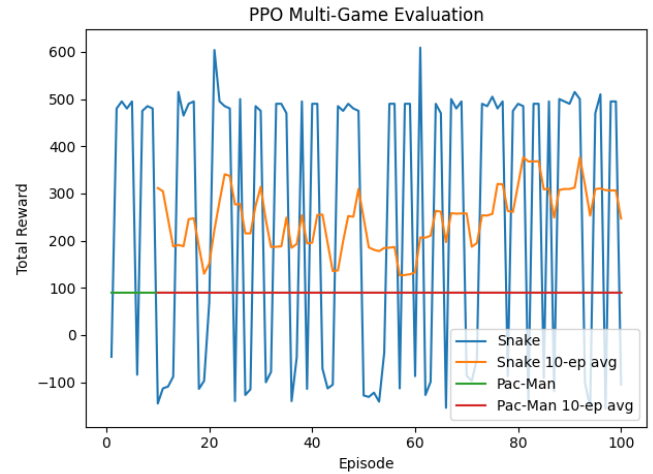


Figure 9. Episode rewards under the final multi-head PPO run (250 K pretrain → 250 K mixed). Blue: Snake raw rewards; green: Snake 10-episode running average. Orange: Pac-Man raw rewards; red: Pac-Man 10-episode running average.

If we’d had more time and compute, we’d focus on three streamlined extensions. First, we’d dramatically lengthen and skew the curriculum—e.g. 500 K steps on Snake alone, then mixed training with five Snake workers to three Pac-Man—to lock in the five-apple loop before Pac-Man ever sees the network. Second, we’d partially decouple the feature extractor by branching the CNN into two “towers” after a few shared layers (or using task-conditioned BatchNorm) so that Snake’s representations aren’t constantly overwritten

by Pac-Man frames. Third, we'd enrich the Pac-Man reward signal with a level-clear bonus and an auxiliary head predicting distance to the nearest pellet, encouraging the agent to explore deeper into the maze instead of looping on the nearest four pellets.

5.2. Q-Learning Approach

In the approach for the Q-learning agent to be trained on both games the reward components are kept as described in Section 4.2 and Section 4.4.

Also as described in Section 4.4 the ghost movement was fixed for this approach.

Results and analysis

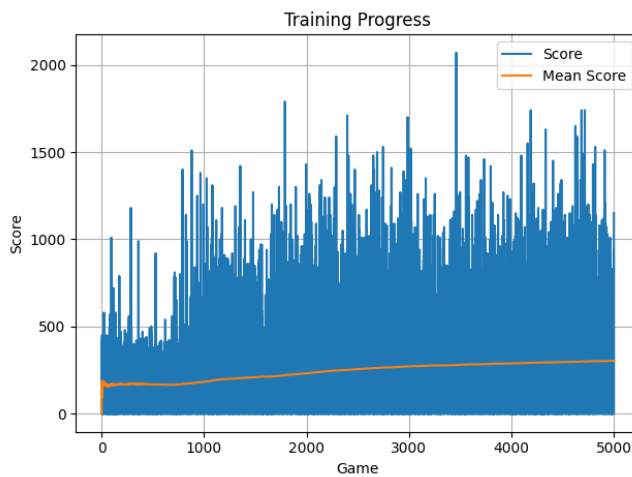


Figure 10. Score in each training game and average training score

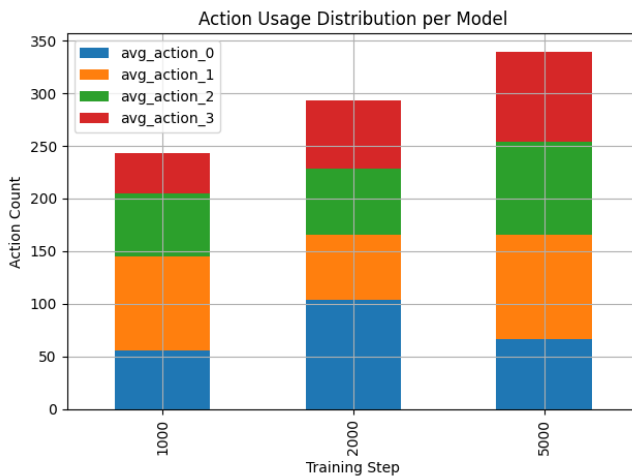


Figure 11. Average action distribution per game in snake over 100 evaluation games



Figure 12. Average action distribution per game in snake over 100 evaluation games

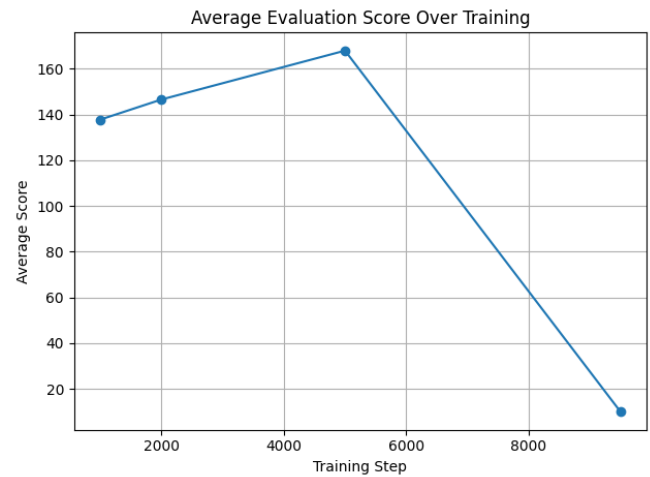


Figure 13. Average score over 100 evaluation games

Training scores are rising:

As Figure 10 demonstrates consistent improvement in training scores over time, indicating that the agent is learning to optimize its behavior in the environment. **Training scores are rising:**

In Figure 11 and Figure 12 is seen that the action distribution is spread evenly. Also the Amount of actions increases which means the agent learns to survive longer

Conclusion

Training for a longer period of time would probably increase the agents performance.

6. Conclusion

In this project, we investigated the development and performance of reinforcement learning agents trained on two distinct games—Snake and Pac-Man—using both Q-learning and Proximal Policy Optimization (PPO). Our experiments showed that while both algorithms can learn effective strategies in single-game settings, PPO consistently outperforms Q-Learning in high-dimensional, sparse-reward environments like Pac-Man due to its superior sample efficiency and stability.

For the combined agent experiments, we found that naïve multi-task training with shared heads leads to destructive interference between tasks, especially when game dynamics differ significantly. A curriculum-based approach showed marginal improvements, but the most promising results came from using a shared feature extractor with separate policy/value heads. This multi-head setup enabled partial task decoupling and improved performance on Pac-Man, though Snake’s performance remained volatile, suggesting interference still occurs at the shared representation level.

To further advance multi-task RL in discrete environments, we recommend:

1. Extended and task-skewed pretraining curricula,
2. Partially branched or task-conditioned architectures to reduce representational interference, and
3. More informative reward shaping, especially for sparse-reward tasks like Pac-Man.

These directions offer promising paths for improving generalization and transfer in multi-game RL agents.

References

- [1] Maxim Lapan. *Deep Reinforcement Learning Hands-On*. Packt Publishing, 2019. 2
- [2] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. 1
- [3] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. In *Advances in Neural Information Processing Systems*, 2017. 1, 2
- [4] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2018. 1
- [5] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3-4):279–292, 1992. 1